React CSR과 현재 Next SSR의 데이터 통신

- 날짜: 2026-10-30
- CSR 대상: `lgcns-fe`. Vite 8, React 19, `react-router-dom`. 진입은 `index.html`의 `<script type="module" src="/src/main.tsx">`
- SSR 대상: `lgcns-fe-next`. Next.js 16.4.3, `cacheComponents: true`, `partialPrefetching: true`
- 두 프로젝트 소스에는 `fetch()`가 없다. 아래 "지금 소스"는 저장소가 실제로 데이터를 옮기는 길이고, "공식 문서의 요청 예"는 이 저장소에 없는 예다.

데이터 통신은 화면이 값을 얻기까지 누가, 언제, 어디로 요청하는가다. CSR은 브라우저가 빈 껍데기를 받은 뒤 모듈을 실행하고, 그 안에서 값을 만든다. 지금 Next 라우트는 서버가 요청 URL을 읽어 HTML에 값을 넣은 뒤 보낸다.

1. 지금 소스에서 값이 오는 곳

	lgcns-fe CSR	**lgcns-fe-next SSR**
첫 HTML	`index.html`의 `<div id="root">`는 비어 있다	서버가 페이지 함수를 실행한 HTML. `/feed?page=1`이면 문단에 `1`이 들어간다
스크립트	`<script type="module">`. 모듈은 파싱을 막지 않고, 문서가 파싱된 뒤 실행된다	페이지 파일에 `<script src>`가 없다. `"use client"` 모듈만 브라우저 번들이다
렌더 시작	`src/main.tsx`의 `createRoot(...).render`	서버 컴포넌트 함수. 기본이 서버다
경로	`BrowserRouter` + `Routes`. 브라우저는 주소창을 읽고 클라이언트에서 `Home`, `About`, `Login`을 고른다	폴더가 경로다. `[blogId]`, `[...productCategoryName]`, `/feed`
목록 데이터	`src/data/socialData.ts` 배열을 `SocialIconList`가 `import`한다. 빌드된 JS 안에 있다	상품·블로그·피드는 URL에서 읽는다. 별도 목록 파일이 없다
사용자 입력	`CountBtn`의 `useState(0)`, `Login`의 `email`·`password` `useState('')`	`MobileNavigation`의 `useState(false)`, `FeedNextButton`의 `onClick`
서버로 나가는 값	`Login`의 `handleSubmit`은 `preventDefault`만 한다. 요청을 보내지 않는다	버튼은 `router.push`로 다음 URL만 연다. 서버는 그 다음 요청의 쿼리를 다시 읽는다

1.1 CSR. 모듈 안의 데이터

`lgcns-fe/index.html`은 앱 마크업을 가지고 있지 않다. 브라우저는 문서를 파싱한 뒤 모듈을 실행한다.

```
createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>,
)
```

소셜 목록은 네트워크 응답이 아니다. `socialData.ts`가 배열을 선언하고, `SocialIconList.tsx`가 그 모듈을 불러 `map`한다.

```
const socialData: socialDataType[] = [
  { id: 1, name: 'Discord', url: 'https://discord.com', icon: DiscordIcon },
]
```

```
{socialData.map((social) => (
  <SocialIconItem key={social.id} social={social} target="_blank" />
))}
```

카운트와 로그인은 브라우저 메모리다. 서버가 이 값을 만들지 않는다.

```
const [count, setCount] = useState<number>(0)
```

```
const [email, setEmail] = useState('')
const [password, setPassword] = useState('')

const handleSubmit = (event: FormEvent<HTMLFormElement>) => {
  event.preventDefault()
}
```

버튼을 누르거나 글자를 넣으면 `useState`가 그 컴포넌트를 브라우저에서 다시 그린다. 새로고침하면 `count`는 `0`, 입력칸은 빈 문자열로 돌아간다. 그 값을 저장하는 요청이 소스에 없다.

1.2 SSR. 요청 URL이 데이터다

`/feed?page=1&limit=10&sort=asc&search=cat`를 열면 브라우저 요청의 쿼리가 `searchParams`다.
`app/(main)/feed/page.tsx`는 동기 함수이고, 프로미스를 `<Suspense>` 안으로 넘긴다.
`component/templates/FeedContent.tsx`가 서버에서 `await`한다.

```
const { page, limit, sort, search } = await searchParams
```

풀린 문자열이 HTML 문단이 되고, `FeedNextButton`에는 문자열이 props로 넘어간다. 버튼 파일만 `"use client"`다. 클릭은 브라우저에서 다음 주소를 연다.

```
router.push(
  `/feed?page=${parseInt(page) + 1}&limit=${limit}&sort=${sort}&search=${search}`
)
```

`page`가 `"1"`이면 다음은 `/feed?page=2&limit=10&sort=asc&search=cat`다. 그 주소는 새 문서 요청이고, 서버가 쿼리를 다시 읽어 HTML을 만든다. 카운트처럼 브라우저 메모리만 바뀌는 갱신이 아니다.

블로그와 상품도 서버가 URL을 읽는다. 쿼리 대신 경로다.

URL	서버가 푸는 값	함수

/blog/name	{ blogId: "name" }	BlogContent의 await params
/product/laptop/gram	{ productCategoryName: ["laptop", "gram"] }	ProductContent의 await params
/feed?page=1&limit=10&sort=asc&search=cat	쿼리 네 문자열	FeedContent의 await searchParams

ProductContent의 console.log는 서버 터미널에 찍힌다. CustomTitle은 그 서버 렌더 중에 호출된다.

MobileNavigation은 Next 안에서도 CSR에 가깝다. "use client"와 useState(false)로 메뉴를 열고, 서버에 열림 여부를 보내지 않는다. 경로 이동은 <Link href="/">다.

2. 요청이 일어나는 시점

단계	CSR lgcns-fe	SSR lgcns-fe-next 피드
1	브라우저가 index.html을 받는다. #root는 비어 있다	브라우저가 /feed?page=1&...를 연다
2	type="module" 스크립트를 받아 실행한다	서버 FeedPage가 <Suspense fallback={ Loading... }>를 반환한다
3	createRoot가 App을 그린다	서버 FeedContent가 await searchParams를 한다
4	socialData는 이미 모듈 안에 있다. count는 0이다	응답 HTML에 page: 1이 포함된다
5	클릭은 setCount다. 추가 HTTP 요청이 없다	Next는 클라이언트 모듈을 보내고 버튼에 이벤트를 붙인다
6	로그인 제출도 preventDefault에서 끝난다	Next 클릭은 router.push로 2번부터 다시 한다

async function FeedContent는 서버에서 실행되는 ECMAScript async 함수다. HTML <script async>가 아니다. "use client"는 그 파일을 브라우저 번들로 보내라는 지시문이다. 피드에서 그 지시문은 버튼 파일에만 있다.

cacheComponents가 켜져 있어 searchParams와 params는 요청 시각에야 아는 값이다. await가 <Suspense> 밖이면 프리렌더가 막힌다. 세 라우트 모두 페이지는 프로미스만 넘기고, 자식이 await한다.

3. 공식 문서의 요청 예

이 절의 fetch는 저장소에 없다. React와 Next 문서가 적은, 원격 데이터를 주고받는 모양이다. 지금 소스의 await searchParams와 나란히 보면 요청의 주체만 다르다.

3.1 브라우저가 그린 뒤 요청하는 CSR

React 문서의 데이터 요청은 컴포넌트가 브라우저에 나타난 뒤 useEffect 안에서 fetch하는 형태다. 첫 HTML에는 목록이 없고, 응답이 도착하면 useState로 다시 그린다.

```
const [posts, setPosts] = useState([])

useEffect(() => {
  let ignore = false
  fetch("https://api.vercel.app/blog")
    .then((response) => response.json())
    .then((data) => {
      if (!ignore) setPosts(data)
    })
  return () => {
    ignore = true
  }
}, [])
```

이 요청의 주체는 브라우저다. 서버 HTML을 받은 다음, 모듈이 실행되고, 그 다음에 HTTP가 나간다. 응답 전의 화면은 빈 배열이다. Effect 정리 함수는 컴포넌트가 사라지거나 의존값이 바뀔 때 늦게 도착한 응답을 무시한다. `lgcns-fe`의 `CountBtn`과 `Login`은 `useState`만 있고 이 `useEffect`와 `fetch`는 없다.

3.2 서버가 HTML을 만들기 전에 요청하는 SSR

Next 문서의 서버 컴포넌트는 함수를 `async`로 두고 `await fetch`한다. 브라우저는 그 요청을 보내지 않는다. 서버가 응답을 받은 뒤 목록이 들어 있는 HTML을 보낸다.

```
export default async function Page() {
  const data = await fetch("https://api.vercel.app/blog")
  const posts = await data.json()
  return (
    <ul>
      {posts.map((post) => (
        <li key={post.id}>{post.title}</li>
      ))}
    </ul>
  )
}
```

문서에 따르면 이 `fetch`는 기본으로 캐시되지 않고, 컴포넌트 트리 안의 같은 요청은 메모이즈된다. 결과가 늦으면 그 컴포넌트가 끝날 때까지 렌더가 기다린다. 문서가 적은 방법은 `<Suspense>`로 그 컴포넌트만 감싸 나머지 껍데기를 먼저 보내는 것이다. 지금 피드의 `await searchParams`가 그 자리와 같다. 기다리는 대상이 외부 API가 아니라 현재 요청의 쿼리다.

클라이언트 컴포넌트에서 원격 데이터를 다루는 Next 문서의 길은 두 가지다. 서버가 프로미스를 만들어 `<Suspense>` 자식에 넘기고 클라이언트가 React `use`로 푸는 길, 그리고 SWR이나 TanStack Query처럼 브라우저가 요청하는 길이다. 지금 `FeedNextButton`은 둘 다 쓰지 않는다. 서버가 이미 푼 문자열만 받는다.

4. 같은 화면을 두 방식으로 두면

`/feed?page=1&limit=10&sort=asc&search=cat`의 네 값을 예로 둔다.

	CSR로 둔다면	지금 SSR
첫 화면	`#root`가 비어 있다가 모듈이 그린다. URL을 읽으려면 브라우저 코드가 `window.location.search`나 라우터 훅을 본다	서버가 쿼리를 읽어 `page: 1`이 들어 있는 HTML을 보낸다
새로고침	브라우저가 다시 모듈을 실행한다. `useState` 초기값으로 돌아간다	브라우저가 같은 URL을 다시 요청하고, 서버가 같은 쿼리로 HTML을 다시 만든다
다음 페이지	클라이언트 상태가 `page`를 1 올리거나, 브라우저가 `fetch`로 다음 목록을 받아 온다	`router.push`가 `/feed?page=2&...`를 연다. 서버가 다시 `await searchParams`한다
비밀값	브라우저 번들에 들어가면 사용자가 볼 수 있다	서버 컴포넌트의 토큰과 쿼리는 클라이언트 번들에 넣지 않는다. 지금 피드에는 토큰이 없다

`lgcns-fe`의 소셜 목록은 1행에 가깝다. 네트워크 대신 모듈 import다. 사용자가 새로고침해도 같은 배열이 번들에 있다. `CountBtn`은 3행의 클라이언트 상태다. 서버가 그 숫자를 모른다.

5. 공식 문서

- React useEffect로 데이터 가져오기
- React Server Components
- React use
- Next.js Fetching Data. 로컬 `node_modules/next/dist/docs/01-app/01-getting-started/06-fetching-data.md`
- Server and Client Components. 로컬 `node_modules/next/dist/docs/01-app/01-getting-started/05-server-and-client-components.md`
- Client-side data fetching
- MDN fetch
- MDN script type=module
- HTML 모듈 스크립트
- useRouter